

0. Introduction to Collections

읽기 전

이 글은 [Java Tutorials - Collections](#) 공식 문서를 Claude 크롬 확장 프로그램의 도움을 받아 번역·정리한 것입니다.

원문 해석 과정에서 부정확하거나 어색한 표현이 있을 수 있으니 참고 부탁드립니다.

0.1 Collection 이란?

여러 개의 원소를 하나의 단위로 묶는 객체로, 컨테이너라고도 불린다.

컬렉션은 데이터를 저장, 조회, 조작, 전달하는 데 사용된다.

이때 대상이 되는 데이터를 집합 데이터라고 표현하며, 하나의 그룹을 이루는 데이터 항목을 다룬다.

ex)

포커 패 -> 카드들의 컬렉션

메일 폴더 -> 편지들의 컬렉션

0.2 Collection Framework

- 컬렉션을 표현하고 조작하기 위한 **통합 아키텍처**
- 개별 자료구조가 아니라, 그것들을 **일관된 방식으로 다루기 위한 전체 설계**를 가리킨다

모든 컬렉션 프레임워크는 다음과 같은 세 가지 구성 요소를 포함한다.

i. Interface : 추상체

컬렉션을 표현하는 추상 데이터 타입으로, **표현의 세부 사항과 독립적으로** 컬렉션을 조작할 수 있게 한다.

ii. Implementation : 구현체

컬렉션 인터페이스의 구체적인 구현으로, 본질적으로 **재사용 가능한 자료구조**이다.

iii. Algorithms : 알고리즘

검색 및 정렬과 같이, 컬렉션 인터페이스를 구현한 객체에 대해 유용한 연산을 수행하는 메서드다.

공식 문서에서는 이 알고리즘을 **다형적(polymorphic)** 이라고 표현한다.

이는 동일한 메서드가 해당 인터페이스의 서로 다른 여러 구현체에 대해 그대로 사용될 수 있음을 의미한다.

0.3 Java Collections Framework 의 이점

문서에서는 여섯 가지 이점을 제시한다.

첫째, 프로그래밍 노력의 절감이다.

유용한 자료구조와 알고리즘을 제공함으로써, 개발자가 저수준의 plumbing 대신 프로그램의 본질적인 부분에 집중하도록 한다.

또한 서로 무관한 API 간의 상호운용성을 촉진하여, 어댑터 객체나 변환 코드를 작성할 필요를 없앤다.

둘째, 프로그램의 속도와 품질 향상이다.

고성능·고품질의 구현체를 제공하며, 각 인터페이스의 여러 구현체는 상호 교체가 가능하다.

따라서 구현체를 교체하는 것만으로 프로그램을 손쉽게 튜닝할 수 있다.

셋째, 무관한 API 간 상호운용성이다.

컬렉션 인터페이스는 API 들이 컬렉션을 주고받는 공통 언어 역할을 한다.

문서의 예시에 따르면, 노드 이름의 컬렉션을 제공하는 네트워크 관리 API 와 컬럼 헤더의 컬렉션을 기대하는 GUI 툴킷은 독립적으로 작성되었더라도 매끄럽게 연동된다.

넷째, 새 API 의 학습·사용 노력 절감이다.

과거에는 API 마다 고유한 임시 컬렉션 서브 API 를 두어 일관성이 없었고, 이를 개별적으로 학습해야 했다. 그러나 표준 컬렉션 인터페이스의 등장으로 이 문제가 해소되었다.

다섯째, 새 API 의 설계 노력 절감이다.

설계자와 구현자는 컬렉션 기반 API 를 만들 때마다 매번 바퀴를 재발명할 필요 없이, 표준 인터페이스를 그대로 사용할 수 있다.

여섯째, 소프트웨어 재사용의 촉진이다.

표준 컬렉션 인터페이스를 준수하는 새 자료구조와, 그 인터페이스를 구현한 객체에 대해 동작하는 새 알고리즘은 본질적으로 재사용 가능하다.